Tutorial 5: Shuffling a set of genomic contacts to form a new set

Problem Statement

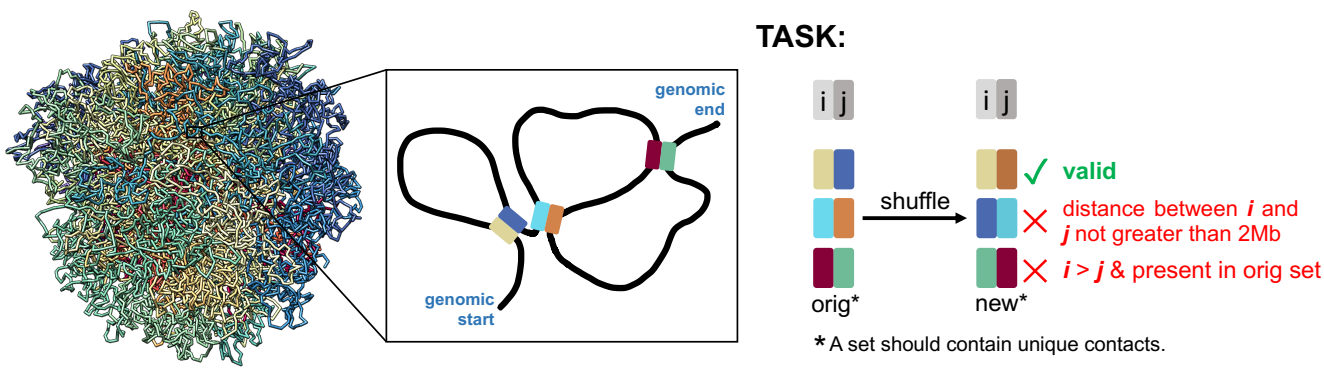
In this tutorial, an example is taken from the field of 3D genome biology to show how Optimus can solve a basic shuffling task given certain conditions.

Mammalian genomes are packaged into a complex three-dimensional (3D) structure inside the nucleus. They are efficiently folded bringing DNA regions near or far from each other into contact, facilitating the tight regulation of replication and gene expression.

We have taken a set of 734 pairs of i and j 40-kilobase DNA regions that are known to be in contact inside a cell. In our set, a 40-kb DNA region is represented as a single integer that is equal to its end position divided by the length of the region, which is 40 kb. For instance, a region, with start and end positions at the 1st and 40000th nucleotides, respectively, is denoted as **1** ($40000th/40000$ bases $= 1$). This simplifies the notation for a contact to a pair of positive integers as shown below

```
##        i   j
## [1,]  96 151
## [2,]  96 174
## [3,] 322 374
## [4,] 309 392
## [5,] 321 392
## [6,] 373 460
```

, where $i \in Z^+$ and $j \in Z^+$.



Take note that our set of contacts complies with the following criteria:

1. The set only contains **unique, long-range contacts**, with i and j regions always separated by greater than 2 megabases/Mb. In terms of integer, the threshold is 50 ($2Mb/40kb = 50$), such that $j - i > 50$.
2. i is set to be always less than j. This avoids duplicate contacts in the set, such as

```
##        i   j
## [1,] 181 273
## [2,] 273 181
```

3. A region can take part in more than one contact (as in region 96 in rows 1 and 2 in the table above), given that the contacts it forms comply with the two aforementioned criteria.

The task is to take the DNA regions from the original set (shown above) and to shuffle them to produce a new set of contacts. This is useful for specific research on genomic 3D contacts requiring good quality negative controls. The challenge here is to maximise the number of new contacts that are valid, meaning that they (A) still satisfy the aforementioned criteria and (B) are not present in the original set.

Defining Optimus Inputs

Now, we solve the problem using Optimus by doing the following:

1. Keeping the original order of i's in the original set (referred hereinafter as IJ.ORIG), randomly shuffle once all j's without replacement. The output will be object K to be supplied to Optimus().

```
K <- IJ.ORIG
# Shuffle all j's once
K[, "j"] <- sample(x = K[, "j"], size = nrow(K), replace = FALSE)
```

2. Define the rest of Optimus inputs:

The $m()$ function should operate on the object K, which is the original set of contacts with all j's shuffled once (from Step 1). It can also accept a supplementary object $DATA$ that holds data necessary for its calculations or comparisons. Output of $m()$ is the observable object O that is the percentage of contacts not satisfying the aforementioned criteria or are already in the original set. To get O, $m()$ will require the following additional data: a. original set of contacts (IJ.ORIG), b. number of contacts, and c. gap threshold set between contacting regions in terms of positive integer (50). These are stored in the $DATA$ object that is supplied to $m()$.

```
DATA <- NULL
DATA$IJ.ORIG <- IJ.ORIG
DATA$gaplimit <- 50
DATA$numContacts <- 173

m <- function(K, DATA){
  # Total number of erroneous contacts in the new set
  errCont <- sum(
    # Contacts not satisfying the threshold value of the gap between their two regions
    ( (K[,"j"]-K[,"i"]) <= DATA$gaplimit )  |
    # Contacts also present in the original set
    ( K[,"j"]==DATA$IJ.ORIG[,"j"]        )  |
    # Duplicated contacts in the new set
    ( duplicated(K)                      )
  )
  # Percentage of erroneous contacts
  O <- (errCont/DATA$numContacts)*100
  return(O)
}
```

Dictated by the $u()$ function, the error percentage, O, will be directly used as the pseudo-energy, E, since we do want to minimise this value. Meanwhile, the K snapshot quality, Q, which we want to improve, can be the negative of O.

```
u <- function(O) {
    RESULT <- NULL
    RESULT$Q <- -O
    RESULT$E <- O
    return(RESULT)
}
```

Finally, $r()$, which defines how object K will be altered at every iteration, takes two j's and then swaps them, without changing the order of their partner i's.

```
r <- function(K) {
    K.new <- K
    # Indices of two randomly chosen j's to swap
```

```
    new.ind <- sample(x = 1:length(K.new[, "j"]), size = 2, replace = FALSE)
    # Swap the j's
    K.new[new.ind, "j"] <- K.new[rev(new.ind), "j"]
    return(K.new)
}
```

3. Perform acceptance ratio annealing and replica exchange Optimus runs.

Acceptance Ratio Annealing Optimus Run

We first use the Acceptance Ratio Annealing Optimus to solve the problem. As in Tutorials 2 and 4, Optimus is set to do 200 000 iterations, 2 annealing cycles and to produce 4 replicas of the same simulations.

```
Optimus(NCPU = 4, K.INITIAL = K, rDEF = r, mDEF = m, uDEF = u, OPT.TYPE = "SA",
    OPTNAME = "IJ.NEW.OPTI.SA", DATA = DATA, NUMITER = 2e+05, CYCLES = 2, DUMP.FREQ = 1e+05,
    LONG = FALSE)
```

Note that if additional data is required by $m()$, that object should be assigned to $DATA$ argument of Optimus(), otherwise $DATA$ defaults to NULL.

The 4 new sets obtained have the following error percentages:

Table 1: OPTIMUS SA: % of Erroneous Contacts

Set	%
1	11.5804
2	11.4441
3	11.7166
4	11.7166

Replica Exchange Optimus Run

Now, we use replica exchange Optimus using 12 cores and the same number of iterations (200 000) as in the acceptance ratio annealing Optimus run. Similarly to Tutorials 2 to 4, STATWINDOW is set to 50 and the acceptance ratios are as follows:

```
ACCRATIO <- c(90, 82, 74, 66, 58, 50, 42, 34, 26, 18, 10, 2)

Optimus(NCPU = 12, K.INITIAL = K, rDEF = r, mDEF = m, uDEF = u, ACCRATIO = ACCRATIO,
    OPT.TYPE = "RE", DATA = DATA, OPTNAME = "IJ.NEW.OPTI.RE", NUMITER = 2e+05,
    STATWINDOW = 50, DUMP.FREQ = 1e+05, LONG = FALSE)
```

Table 2: OPTIMUS RE: % of Erroneous Contacts

Set	Replica Acceptance Ratio	%
1	90	18.3924
2	82	21.5259
3	74	21.1172
4	66	20.2997
5	58	20.1635
6	50	17.9837
7	42	18.9373
8	34	24.1144
9	26	21.3896
10	18	21.7984

Set	Replica Acceptance Ratio	%
11	10	17.8474
12	2	19.7548

Notably, none of the 12 new sets of contacts from the replica exchange run are better than the sets produced by the acceptance ratio annealing Optimus.

Summary

We have added yet another one to the diverse applications of Optimus, this time by performing a basic shuffling of pairs of positive integers that, in this case, represent contacts between DNA regions, to form a new set of contacts. Additionally, Optimus showed to be a platform, where certain restrictions or conditions can easily be incorporated to a given task, which usually is required for specific research objectives. The table below shows the % of erroneous contacts of the best outputs from the acceptance ratio annealing and replica exchange Optimus runs, with the former producing the better set with 650 valid contacts out of the 734 total number of original contacts.

Table 3: Best Outputs: % of Erroneous Contacts

Method	%
Optimus (Acceptance Ratio Annealing)	11.4441
Optimus (Replica Exchange)	17.8474